

Trabalho Final - Estrutura de Dados I 2026-1

Sistema de Controle de Biblioteca

Descrição Geral

O trabalho final do CCR Estrutura de Dados consistirá no desenvolvimento de um sistema de controle de biblioteca. O sistema deverá permitir o cadastro e gerenciamento de livros e usuários.

Cada usuário deve conter:

- email
- Nome

Cada livro deve conter:

- Título
- Autor
- Ano de publicação
- Código (ID único)
- Status (disponível ou emprestado)
- email do usuário que emprestou o livro

Como a entrada de valores alfanuméricos (string) em C gera problemas no **scanf**, considere que todos os valores alfanuméricos não conterão espaços. Caso queira eliminar esta limitação, procure funções como **fgets** para entradas de strings com espaço.

Funcionalidades (Menu)

O sistema deverá apresentar um menu com as seguintes opções:

1. Cadastro
 2. Consulta
 3. Atualização
 4. Exclusão
 5. Empréstimo
 6. Devolução
 0. Sair
-

Descrição das opções

1- Cadastro

Acessa a opção de cadastros. Aparece um novo menu com as seguintes opções:

1. Livros
2. Usuários
0. Voltar

Para cada opção pedir os dados necessários (conforme definido na struct criada). O cadastro não deve permitir duplicações, ou seja, o código (ID) do livro deve ser único e o email do usuário também. O código do livro ID pode ser sequencial e implementado de forma que o usuário não precise digitá-lo. Caso seja informado um código ou email já

existente, o sistema deve informar. Para o status do livro, você pode usar o código 0 (disponível) ou 1 (emprestado). No caso do cadastro, não precisa pedir o status já que ele será, por padrão, 0.

2- Consulta

Acessar a opção de consulta. Aparece um novo menu com as seguintes opções:

1. Livros
2. Usuários
3. Empréstimos
0. Voltar

Permite buscar livros e usuários cadastrados:

Livros:

- Por código (retorna um livro específico)
- Por autor (retorna todos os livros daquele autor)

Caso não encontre:

Exibir: Livro não encontrado

Usuários:

- Por email
- Por nome

Caso não encontre:

Usuário não cadastrado

Empréstimos:

Pede o email de um usuário e retorna, caso existir, os livros em posse do usuário

3- Atualização

Acessar a opção de consulta. Aparece um novo menu com as seguintes opções:

1. Livros
2. Usuários
0. Voltar

Permite alterar:

- Título (Livros)
- Autor (Livros)
- Ano (Livros)
- Nome (Usuário)

Perceba que, para alterar, o usuário deve informar os dados do objeto a ser alterado. Para livros o ID e para usuários o email.

4- Exclusão

Mesmo raciocínio das opções anteriores: o usuário escolhe se quer excluir um livro ou usuário. Para o livro, informa o código, para o usuário o email. Informar se a exclusão foi realizada ou não.

5- Empréstimo

O usuário informará o código do livro a ser emprestado, se o status estiver como disponível, o processo continua. Em seguida, pede o email do usuário. Lembre-se de alterar o status do livro para emprestado e atualizar o email do usuário que fez o empréstimo.

6- Devolução

O usuário informará o código do livro a ser devolvido, se o status estiver como emprestado, o processo continua e atualiza os campos status para disponível e o email do usuário para vazio. Informar caso o livro não estava emprestado.

0 - Sair

Finaliza o programa e limpar a memória alocada.

Instruções:

- Deverá ser utilizada alocação dinâmica. Você escolhe a melhor estratégia: lista simples, duplas, filas, árvores, etc.
- Informar, como comentário no código, o uso de LLM para gerar parte do código: a razão do uso e onde foi usado.

Extras:

- Usar uma estrutura auxiliar (árvore binária) para cadastrar os ID dos livros e verificar de forma mais rápida se o código já existe.
- Manter a lista de livros ou usuários ordenadas (código/título ou email)
- Estiver no Git com commits

Apresentação:

- A apresentação é individual, mas o trabalho pode ser realizado em duplas. As notas são individuais, ou seja, uma dupla pode receber notas diferentes. A nota considera tanto o trabalho quanto as respostas do apresentador.
- A avaliação do trabalho considerará:
 - A organização do código (10%)
 - O uso de funções de forma coerente (20%)
 - Comentários em partes importantes do código. Evite comentários desnecessários (10%)
 - Atende os requisitos do trabalho (60%)